

# PayTracka

## Technical Documentation

Version 1.0 — Buildathon MVP

|                |                                                              |
|----------------|--------------------------------------------------------------|
| Live URL       | https://paytracka.lovable.app                                |
| Frontend       | React 18 + TypeScript + Tailwind CSS                         |
| Backend        | Supabase (PostgreSQL + Auth)                                 |
| AI Integration | Anthropic Claude API (claude-sonnet-4-20250514)              |
| Payment API    | Interswitch Web Checkout + BVN Identity + Transaction Search |
| Deployment     | Lovable (React) + Railway (Node.js/FastAPI backends)         |

### 1. System Architecture

PayTracka uses a three-layer architecture: a React frontend deployed on Lovable, a Supabase backend for authentication and database, and two optional Node.js/FastAPI microservices for Interswitch payment processing and analytics.

| Layer           | Technology                       | Purpose                                                               |
|-----------------|----------------------------------|-----------------------------------------------------------------------|
| Frontend        | React 18 + TypeScript + Tailwind | All 7 screens, state management, API calls                            |
| Authentication  | Supabase Auth                    | Email/password login, Google OAuth, session management                |
| Database        | Supabase PostgreSQL              | Invoices, users, payment records                                      |
| AI Engine       | Anthropic Claude API             | Reminder messages, risk prediction, cash flow summary, invoice writer |
| Payment API     | Interswitch API                  | Payment links, BVN verification, transaction search                   |
| Node.js Service | Express.js                       | Interswitch webhook handler, payment link generation                  |
| Python Service  | FastAPI                          | Insights calculations, overdue detection scheduler                    |

### 2. Database Schema

## 2.1 users table

```
CREATE TABLE users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  full_name   VARCHAR(255) NOT NULL,  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  phone       VARCHAR(20),  
  bvn_verified BOOLEAN DEFAULT FALSE,  
  bvn_verified_at TIMESTAMP,  
  created_at  TIMESTAMP DEFAULT NOW()  
);
```

## 2.2 invoices table

```
CREATE TABLE invoices (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  invoice_ref VARCHAR(20) UNIQUE NOT NULL,  
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,  
  client_name VARCHAR(255) NOT NULL,  
  client_email VARCHAR(255),  
  amount      NUMERIC(12,2) NOT NULL,  
  due_date    DATE NOT NULL,  
  payment_method VARCHAR(50) DEFAULT 'pay_link',  
  status      VARCHAR(20) DEFAULT 'pending',  
  description TEXT,  
  payment_link TEXT,  
  payment_ref VARCHAR(255),  
  paid_at     TIMESTAMP,  
  created_at  TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_invoices_user_id ON invoices(user_id);  
CREATE INDEX idx_invoices_status  ON invoices(status);  
CREATE INDEX idx_invoices_due_date ON invoices(due_date);
```

### 3. Environment Variables

**Security:** Never commit .env files to GitHub. All secrets must be stored in environment variables only.

```
# Supabase
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your_supabase_anon_key

# Anthropic Claude API
ANTHROPIC_API_KEY=sk-ant-your-key-here

# Interswitch - Web Checkout + Payments
# From: business.quickteller.com -> Developer Tools
ISW_CLIENT_ID=IKIA_your_client_id
ISW_SECRET_KEY=your_secret_key
ISW_MERCHANT_CODE=MX0000
ISW_PAY_ITEM_ID=000000
ISW_BASE_URL=https://sandbox.interswitchng.com
ISW_WEBPAY_URL=https://newwebpay.qa.interswitchng.com

# Interswitch - BVN Identity
# From: developer.interswitchgroup.com -> API Marketplace
ISW_IDENTITY_CLIENT_ID=IKIA_identity_client_id
ISW_IDENTITY_SECRET_KEY=identity_secret_key
ISW_IDENTITY_URL=https://sandbox.interswitchng.com/api/v1/identity
```

### 4. Frontend — React App

#### 4.1 Folder Structure

```
src/
  components/
    Dashboard.tsx      # Home screen with hero card
    Inbox.tsx          # Unified payment inbox
    WhoOwesMe.tsx      # Kanban invoice tracker
    Insights.tsx       # Analytics + AI features
    Profile.tsx        # User profile + settings
    NewInvoiceModal.tsx # Invoice creation form
    ReminderSheet.tsx  # AI reminder generator
  lib/
    supabase.ts        # Supabase client config
    interswitchApi.ts  # Interswitch API calls
    claudeApi.ts       # Anthropic Claude API calls
    currencyUtils.ts   # NGN/KES/ZAR conversion
    greetingUtils.ts   # Time-based greeting logic
  types/
    invoice.ts         # Invoice type definitions
    user.ts            # User type definitions
  App.tsx
  main.tsx
```

#### 4.2 Time-Based Greeting Logic

```
// lib/greetingUtils.ts
export const getGreeting = (): string => {
  const hour = new Date().getHours();
```

```
if (hour >= 5 && hour < 12) return 'Good morning';
if (hour >= 12 && hour < 17) return 'Good afternoon';
if (hour >= 17 && hour < 21) return 'Good evening';
return 'Good night';
};

// Usage in Dashboard.tsx
const greeting = `${getGreeting()}, ${user.full_name.split(' ')[0]}`;
// Output: 'Good morning, Oluchi' or 'Good afternoon, Tolu'
// Works for any timezone automatically using device local time
```

## 4.3 Currency Conversion

```
// lib/currencyUtils.ts
export const convertCurrency = (amount: number, currency: string): string => {
  const rates = { NGN: 1, KES: 3.0, ZAR: 0.51 };
  const symbols = { NGN: 'N', KES: 'KES', ZAR: 'R' };
  const converted = amount * rates[currency];
  return `${symbols[currency]}${converted.toLocaleString()}`;
};
```

## 5. AI Integration — Anthropic Claude API

**[AI:** PayTracka integrates 4 AI features powered by the Anthropic Claude API (claude-sonnet-4-20250514).

### 5.1 AI Reminder Message Generator

Generates a personalised, context-aware payment reminder message based on the client's payment history and invoice details.

```
// lib/claudeApi.ts
export const generateReminderMessage = async (
  clientName: string,
  invoiceRef: string,
  amount: number,
  daysOverdue: number
): Promise<string> => {
  const response = await fetch('https://api.anthropic.com/v1/messages', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      model: 'claude-sonnet-4-20250514',
      max_tokens: 1000,
      messages: [{
        role: 'user',
        content: `Write a professional payment reminder for a freelancer.
          Client: ${clientName}, Invoice: ${invoiceRef},
          Amount: N${amount}, Days overdue: ${daysOverdue}.
          Include [PAYMENT_LINK] placeholder. Under 60 words. Polite but firm.`
      }]
    })
  });
  const data = await response.json();
  return data.content[0].text;
};
```

### 5.2 AI Payment Risk Prediction

Analyses client payment history and returns a risk level (LOW/MEDIUM/HIGH) with a plain-English prediction.

```
export const predictPaymentRisk = async (clientData: ClientData) => {
  const response = await fetch('https://api.anthropic.com/v1/messages', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      model: 'claude-sonnet-4-20250514',
      max_tokens: 1000,
      messages: [{
        role: 'user',
        content: `Analyze payment risk. Client: ${clientData.name}.
          Total invoices: ${clientData.total}, Paid on time: ${clientData.paidOnTime},
          Avg days late: ${clientData.avgDaysLate}.
          Return JSON: { risk_level, prediction, advice }`
      }]
    })
  });
  const data = await response.json();
  return JSON.parse(data.content[0].text);
};
```

## 5.3 AI Cash Flow Summary

Generates a 2-sentence natural language summary of the freelancer's current financial position.

```
export const generateCashFlowSummary = async (financialData: FinancialData) => {
  const response = await fetch('https://api.anthropic.com/v1/messages', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      model: 'claude-sonnet-4-20250514',
      max_tokens: 1000,
      messages: [{
        role: 'user',
        content: `Write a 2-sentence cash flow summary for an African freelancer.
          Earned: N${financialData.earned}, Pending: N${financialData.pending},
          Overdue: N${financialData.overdue}. Max 40 words. Start with
          'You are on track...' or 'Watch out...' based on situation.`
      }]
    })
  });
  const data = await response.json();
  return data.content[0].text;
};
```

## 5.4 AI Invoice Description Generator

Generates a professional invoice description from a simple service type prompt.

```
export const generateInvoiceDescription = async (
  serviceType: string,
  clientName: string,
  amount: number
): Promise<string> => {
  const response = await fetch('https://api.anthropic.com/v1/messages', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      model: 'claude-sonnet-4-20250514',
      max_tokens: 1000,
      messages: [{
        role: 'user',
        content: `Write a professional invoice description.
          Service: ${serviceType}, Client: ${clientName}, Amount: N${amount}.
          1-2 sentences. Professional tone. No greetings.`
      }]
    })
  });
  const data = await response.json();
  return data.content[0].text;
};
```

## 6. Interswitch API Integration

### 6.1 Authentication — OAuth 2.0

All Interswitch API calls require a Bearer token obtained via OAuth 2.0. The token is cached and refreshed before expiry.

```
// lib/interswitchApi.ts
let cachedToken: string | null = null;
let tokenExpiry: number | null = null;

export const getAccessToken = async (): Promise<string> => {
  if (cachedToken && Date.now() < tokenExpiry!) return cachedToken;
  const credentials = btoa(`${ISW_CLIENT_ID}:${ISW_SECRET_KEY}`);
  const res = await fetch(`${ISW_BASE_URL}/passport/oauth/token`, {
    method: 'POST',
    headers: {
      Authorization: `Basic ${credentials}`,
      'Content-Type': 'application/x-www-form-urlencoded'
    },
    body: 'grant_type=client_credentials'
  });
  const data = await res.json();
  cachedToken = data.access_token;
  tokenExpiry = Date.now() + (data.expires_in - 300) * 1000;
  return cachedToken;
};
```

### 6.2 Payment Link Generation

**Important:** Amount MUST be in kobo — multiply Naira by 100. NGN 80,000 = 8,000,000 kobo.

```
export const generatePaymentLink = (invoice: Invoice): string => {
  const params = new URLSearchParams({
    merchant_code: ISW_MERCHANT_CODE,
    pay_item_id: ISW_PAY_ITEM_ID,
    txn_ref: `PT-${invoice.id.slice(0,8)}-${Date.now()}`,
    amount: String(invoice.amount * 100), // kobo
    currency: '566', // NGN
    cust_email: invoice.client_email,
    cust_name: invoice.client_name,
    pay_item_name: invoice.description || 'Invoice Payment',
    site_redirect_url: `${APP_URL}/payment/callback`,
  });
  return `${ISW_WEBPAY_URL}/collections/w/pay?${params}`;
};
```

### 6.3 BVN Verification

```
export const verifyBVN = async (bvn: string): Promise<boolean> => {
  const token = await getAccessToken();
  const res = await fetch(`${ISW_IDENTITY_URL}/bvn`, {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${token}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ bvn })
  });
};
```

```
const data = await res.json();
return data?.verified === true;
// Raw BVN is never stored or returned to the frontend
};
```

## 6.4 Payment Confirmation

After a client pays, Interswitch redirects to the callback URL. The backend verifies the transaction server-side before updating the invoice.

```
// Always verify server-side - never trust redirect params alone
export const verifyPayment = async (txnRef: string, amount: number) => {
  const token = await getAccessToken();
  const res = await fetch(
    `https://qa.interswitchng.com/collections/api/v1/gettransaction.json`,
    {
      params: { merchantcode: ISW_MERCHANT_CODE, transactionreference: txnRef, amount },
      headers: { Authorization: `Bearer ${token}` }
    }
  );
  const data = await res.json();
  // ResponseCode '00' = Approved
  return data.ResponseCode === '00';
};
```



## 7. Non-Functional Requirements

| ID     | Requirement      | Specification                                                         |
|--------|------------------|-----------------------------------------------------------------------|
| NFR-01 | Load time        | All screens load in under 2 seconds                                   |
| NFR-02 | Offline support  | Supabase cache enables offline viewing of invoices                    |
| NFR-03 | Payment security | Server-side Interswitch verification before any invoice status update |
| NFR-04 | BVN privacy      | Raw BVN never stored in database or returned to frontend              |
| NFR-05 | Secrets          | All API keys in environment variables, never in source code           |
| NFR-06 | Amount handling  | Stored in Naira in DB, converted to kobo only for Interswitch calls   |
| NFR-07 | Multi-currency   | NGN/KES/ZAR toggle using approximate conversion rates                 |
| NFR-08 | Error handling   | All API failures show user-friendly messages, never raw errors        |

## 8. Deployment

| Service           | Platform       | URL                                                                                 |
|-------------------|----------------|-------------------------------------------------------------------------------------|
| React Frontend    | Lovable        | <a href="https://paytracka.lovable.app">https://paytracka.lovable.app</a>           |
| Supabase Database | Supabase Cloud | Auto-provisioned via Lovable integration                                            |
| Node.js Backend   | Railway        | <a href="https://paytracka-auth.railway.app">https://paytracka-auth.railway.app</a> |
| FastAPI Backend   | Railway        | <a href="https://paytracka-api.railway.app">https://paytracka-api.railway.app</a>   |

---

*This technical documentation covers the PayTracka MVP buildathon submission. The live app is deployed at <https://paytracka.lovable.app>.*